

Software Requirements Specification

Defines the functional and non-functional requirements for the Smart Hire platform.

Program	Diploma in Advanced AI & Software Engineering
Batch	25/26
Project	Smart Hire — Service Marketplace
Team	Nilan Iddawela, Janaka Eranda, Dasun Anjana, Damith Lasantha, Matheesha Milan
Date	21 July 2026

1. Introduction

1.1 Purpose

This document describes the software requirements for **Smart Hire**, a centralized digital marketplace connecting customers with skilled service providers (plumbing, electrical work, home maintenance, tutoring, cleaning, repairs, and technical support). It is intended for the development team, module assessors, and anyone evaluating the system's scope during the viva examination.

1.2 Scope

Smart Hire lets customers discover, compare, and book verified service providers; lets providers publish services, manage availability, and respond to booking requests; and gives administrators tools to moderate the marketplace. The system includes an integrated AI assistant that recommends services, answers questions, and can check availability or create/cancel bookings on a customer's behalf.

1.3 Definitions and Acronyms

TERM	MEANING
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token, used for authenticated sessions
MCP	Model Context Protocol — a standard that lets external AI clients call the same booking/search tools as the in-app assistant
Customer	End user who browses and books services
Provider	End user who publishes and delivers services
Admin	Platform operator who moderates users, services, reviews, and payments

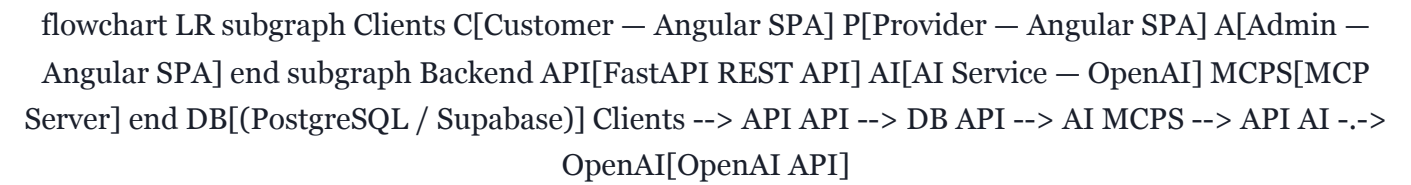
1.4 Overview

Section 2 describes the product at a high level. Section 3 lists functional requirements by module. Section 4 covers interfaces. Section 5 covers non-functional requirements.

2. Overall Description

2.1 Product Perspective

Smart Hire is a new, self-contained system: an Angular single-page frontend, a Python FastAPI backend serving a REST API, and a PostgreSQL database (hosted on Supabase). It integrates the OpenAI API for its AI assistant, and exposes a standalone MCP server so external MCP-compatible clients can use the same booking/search tools.



2.2 Product Functions (Summary)

- Account registration and role-based login (customer / provider / admin)
- Service publishing, browsing, search, and category filtering
- Booking lifecycle: request, accept/reject, complete, cancel
- Reviews and ratings, tied to completed bookings only
- Simulated payments tied to completed bookings
- Notifications for booking, review, and message activity
- Direct messaging between a customer and provider on a shared booking
- AI assistant: service recommendations, conversational booking/availability actions, and provider review summarization
- Admin moderation: users, services, categories, reviews, and payments, plus platform analytics

2.3 User Classes and Characteristics

USER CLASS	DESCRIPTION
Customer	Searches for and books services; manages bookings, favourites, reviews, payments, and messages; uses the AI assistant.
Provider	Publishes services, sets availability and coverage areas, responds to booking requests, messages customers, and reviews AI-generated summaries of their own reviews.
Admin	Moderates users (status, verification), services, categories, and reviews; manages payment records; views platform-wide analytics.

2.4 Operating Environment

LAYER	TECHNOLOGY
Frontend	Angular (standalone components, signal-based state)
Backend	Python, FastAPI, SQLAlchemy ORM
Database	PostgreSQL, hosted on Supabase
AI	OpenAI API (chat completions with native tool-calling); standalone MCP server (Python <code>mcp</code> SDK, stdio transport)
Auth	JWT bearer tokens; passwords hashed with bcrypt

2.5 Design and Implementation Constraints

- The frontend runs without `zone.js` ; components that mutate state inside HTTP callbacks must trigger change detection explicitly.
- The AI assistant must never fabricate a provider, price, availability slot, or booking that isn't backed by real data or a real tool call.
- All money-moving and booking-state actions require an authenticated, active-status user with the correct role.

2.6 Assumptions and Dependencies

- Users have a modern web browser and internet access.
- The AI assistant depends on the OpenAI API being reachable; it degrades to a safe fallback message if not.
- Payments are simulated (no real external payment gateway is integrated).

3. System Features (Functional Requirements)

3.1 Authentication & Account Management

- **REQ-1.1** The system shall let a user register as a customer or provider with email, password, full name, phone, and (for providers) a bio and years of experience.
- **REQ-1.2** The system shall let a registered user log in and receive a signed JWT bearer token.
- **REQ-1.3** The system shall block login and all write actions for suspended or deactivated accounts, with a clear reason shown to the user.
- **REQ-1.4** The system shall let a user view and update their own profile, including an optional password reset.

3.2 Service Marketplace

- **REQ-2.1** The system shall let a provider create, edit, and remove their own service listings (title, description, price, city, duration, category, status).
- **REQ-2.2** The system shall let a customer browse active services, filter by category and city, sort by rating/price/recommended, and search by keyword.
- **REQ-2.3** The system shall let a customer save and remove services as favourites.
- **REQ-2.4** The system shall display whether a provider is verified, based on the admin verification workflow.

3.3 Booking Management

- **REQ-3.1** The system shall let a customer request a booking for an active service with a future date/time and optional notes.
- **REQ-3.2** The system shall let a provider accept or reject a pending booking request.
- **REQ-3.3** The system shall let a customer cancel a pending or accepted booking, and a provider mark an accepted booking complete.
- **REQ-3.4** The system shall notify the other party whenever a booking's status changes.
- **REQ-3.5** The system shall let a provider define weekly availability windows, and let the AI assistant or a direct check confirm real open slots against existing bookings.

3.4 Reviews & Ratings

- **REQ-4.1** The system shall let a customer leave exactly one rating (1–5) and comment per completed booking.
- **REQ-4.2** The system shall recalculate a provider's average rating and review count whenever a new review is created.
- **REQ-4.3** The system shall let a provider generate an AI-written summary of their own service's written reviews.

3.5 Payments

- **REQ-5.1** The system shall let a customer submit a simulated payment for a completed booking, once only, generating a transaction id.
- **REQ-5.2** The system shall let a customer and provider view their own payment history, and let an admin view and update any payment's status.

3.6 Notifications & Messaging

- **REQ-6.1** The system shall notify a user of booking, review-eligible, and message events, with a read/unread state.

- **REQ-6.2** The system shall let a customer and the provider on a shared booking exchange direct messages, restricted to that booking's two participants.

3.7 AI Assistant

- **REQ-7.1** The system shall recommend a service category and matching active listings from a free-text customer description.
- **REQ-7.2** The system shall let a customer chat conversationally with an assistant that is aware of the current date/time, real active listings, and the customer's own bookings.
- **REQ-7.3** The assistant shall be able to check a provider's real availability, create a real booking, and cancel a real booking, only after explicit customer confirmation.
- **REQ-7.4** The system shall expose the same booking/search tools to external MCP clients via a standalone MCP server.

3.8 Administration

- **REQ-8.1** The system shall let an admin view and change a user's account status and verification state.
- **REQ-8.2** The system shall let an admin manage service categories, moderate service listings, and remove reviews.
- **REQ-8.3** The system shall let an admin view real platform statistics (users, bookings, services, revenue) and a real monthly revenue/bookings chart.

4. External Interface Requirements

4.1 User Interfaces

Three role-specific web workspaces (Customer, Provider, Admin), each with a persistent sidebar and a shared top navigation bar (notifications, profile). A floating AI chat widget is available on every customer and provider page.

4.2 API Interface

All backend functionality is exposed as a JSON REST API under `/api`, secured with JWT bearer tokens. See the *API Documentation* deliverable for the full endpoint reference.

4.3 Software Interfaces

- OpenAI Chat Completions API (AI assistant, recommendations, review summaries)
- PostgreSQL (Supabase-hosted) via SQLAlchemy

5. Non-Functional Requirements

5.1 Security

- Passwords are hashed with bcrypt; never stored or returned in plain text.
- All state-changing endpoints require a valid JWT and an active account status.
- Role-based access control is enforced server-side on every endpoint, not just hidden in the UI.
- A user can only read or act on bookings, messages, reviews, and payments they are a genuine party to.

5.2 Performance

Interactive pages should respond within 1–2 seconds under normal load. AI chat replies typically return within a few seconds, bounded by the upstream OpenAI API.

5.3 Reliability

The AI assistant degrades to a safe, honest fallback message if the OpenAI API is unreachable, rather than failing the whole page.

5.4 Usability

Each role has a dedicated, uncluttered workspace; destructive actions (cancel, delete) require explicit confirmation; toast messages confirm the outcome of key actions.

5.5 Maintainability

The backend follows a consistent repository → service → router layering per module. The frontend uses standalone Angular components and a shared design system across customer, provider, and admin workspaces.